

UNIVERSIDAD DEL NORTE — SEDE CAACUPÉ

FACULTAD DE INGENIERÍA INFORMÁTICA

PROGRAMACIÓN III

TRABAJO PRÁCTICO

Desarrollo de un Sistema Web Full-Stack de Pedidos para M.L. Creaciones & Regalos utilizando React, TypeScript, Node.js y SQLite

Profesor:
Carlos A. Benítez B.

Integrantes:
[Integrante 1]
[Integrante 2]
[Integrante 3]

Quinto Semestre

Caacupé – Paraguay
2026

UNIVERSIDAD DEL NORTE — SEDE CAACUPÉ
FACULTAD DE INGENIERÍA INFORMÁTICA
PROGRAMACIÓN III — TRABAJO PRÁCTICO

Desarrollo de un Sistema Web Full-Stack de Pedidos para M.L. Creaciones & Regalos utilizando React, TypeScript, Node.js y SQLite

Autor 1: [nombre] — *Autor 2:* [nombre] — *Autor 3:* [nombre]
Universidad del Norte, Sede Caacupé | [correos institucionales]

***Abstract* — This paper describes the evolution of the order management system of M.L. Creaciones & Regalos, a small personalized-gifts business in Paraguay, from a browser-only prototype into a full-stack web application. The frontend was rebuilt with React 19 and TypeScript on Vite, keeping the original Bootstrap 5 visual identity, and a REST API was developed with Node.js, Express 5 and the built-in SQLite engine of Node 24. The system provides user accounts with e-mail verification codes, password recovery, role-based access (public, customer and administrator), server-side orders with transactional stock control, order status tracking, automatic e-mail notifications and a product management panel with image upload. Passwords are stored with bcrypt, sessions use JWT in httpOnly cookies, and every input is validated with Zod. The result is a functional system that solves the main limitation of the first version: orders and stock are now shared by every user instead of living in a single browser.**

***Keywords* — Full-Stack, React, TypeScript, Node.js, Express, SQLite, REST API, Authentication, E-mail Verification, Order Management.**

***Resumen* — Este trabajo describe la evolución del sistema de pedidos de M.L. Creaciones & Regalos, microempresa paraguaya de regalos personalizados, desde un prototipo que funcionaba solo en el navegador hacia una aplicación web full-stack. El frontend se reconstruyó con React 19 y TypeScript sobre Vite, conservando la identidad visual con Bootstrap 5, y se desarrolló una API REST con Node.js, Express 5 y el motor SQLite incluido en Node 24. El sistema ofrece cuentas de usuario con código de verificación por correo, recuperación de contraseña, acceso por roles (público, cliente y administrador), pedidos en el servidor con control transaccional de stock, seguimiento del estado de cada pedido, avisos automáticos por correo y un panel de gestión de productos con carga de imágenes. Las contraseñas se guardan con bcrypt, las sesiones usan JWT en cookies httpOnly y todos los datos de entrada se validan con Zod. El resultado resuelve la principal limitación de la primera versión: los pedidos y el stock ahora son compartidos por todos los usuarios en lugar de vivir en un único navegador.**

***Palabras clave* — Full-Stack, React, TypeScript, Node.js, Express, SQLite, API REST, Autenticación, Verificación por Correo, Gestión de Pedidos.**

I. Introducción

Las pequeñas empresas del rubro de regalos personalizados necesitan registrar pedidos, controlar su stock y mantenerse en contacto con sus clientes sin recurrir a sistemas empresariales costosos. M.L. Creaciones & Regalos digitalizó este proceso en una primera versión construida con HTML5, Bootstrap 5 y JavaScript, que guardaba los pedidos en el `localStorage` del navegador.

Esa solución tenía una limitación de fondo: cada navegador guardaba sus propios datos. Un pedido hecho por un cliente desde su teléfono nunca llegaba a la dueña del negocio, el stock no era compartido y el panel con la agenda de pedidos quedaba a la vista de cualquier visitante. Para que el sistema sirva a clientes reales hace falta un servidor que centralice la información y controle quién puede ver o modificar cada dato.

El objetivo de este trabajo es transformar el prototipo en una aplicación web **full-stack**: un frontend en React y TypeScript que consume una API REST propia, desarrollada en Node.js con una base de datos SQLite. Sobre esa base se implementan cuentas de usuario con verificación por correo electrónico, tres módulos con permisos diferenciados (tienda pública, cliente y administrador), pedidos con seguimiento de estado, avisos automáticos por correo y la gestión de productos desde un panel.

II. Marco Teórico

A. Arquitectura cliente-servidor y API REST

En la arquitectura cliente-servidor el navegador (cliente) presenta la interfaz y el servidor concentra los datos y las reglas del negocio. Ambos se comunican mediante una API REST: el cliente envía peticiones HTTP (`GET`, `POST`, `PATCH`, `PUT`, `DELETE`) a rutas que representan recursos —productos, pedidos, presupuestos— y el servidor responde en formato JSON. Esta separación permite validar en el servidor todo lo que llega del cliente, que nunca es confiable.

B. React y TypeScript

React es una biblioteca para construir interfaces a partir de **componentes** reutilizables, que se vuelven a dibujar automáticamente cuando cambia su estado [1]. TypeScript agrega tipos estáticos a JavaScript, lo que permite detectar errores —un campo mal escrito, un dato del tipo incorrecto — al compilar en lugar de en el navegador del usuario [2]. Para la navegación entre pantallas se usó React Router, y para compartir la sesión y el catálogo entre componentes, la Context API de React.

C. Vite y Bootstrap 5

Vite es la herramienta de desarrollo y empaquetado del frontend: sirve la aplicación con recarga instantánea durante el desarrollo y genera los archivos optimizados para producción [3]. Su servidor incluye un *proxy* que reenvía las rutas `/api` al backend, de modo que el navegador ve un único origen. Bootstrap 5.3 [4], ahora instalado como dependencia, mantiene la grilla responsiva y los

componentes visuales de la primera versión, complementados con variables CSS para los colores de la marca.

D. Node.js, Express y SQLite

Node.js permite ejecutar JavaScript (y TypeScript, mediante `tsx`) en el servidor. Express 5 organiza la API en *routers* y *middlewares* —funciones que se ejecutan antes de cada ruta, como la verificación de sesión— y captura automáticamente los errores de las funciones asíncronas [6]. Como base de datos se usó SQLite a través del módulo `node:sqlite` incluido en Node 24 [5], [7]: guarda toda la información en un único archivo, no requiere instalar un servidor de base de datos y soporta transacciones ACID.

E. Autenticación y sesiones

Las contraseñas nunca se guardan en texto plano: se almacenan con **bcrypt**, un algoritmo de *hash* lento y con sal, recomendado para contraseñas [9]. Al iniciar sesión, el servidor emite un **JSON Web Token** (JWT) firmado [8] que viaja en una *cookie* con los atributos `HttpOnly` —el código JavaScript de la página no puede leerla, lo que la protege de ataques XSS— y `SameSite=Lax`, que limita su envío desde otros sitios [10].

F. Correo electrónico y validación de datos

El envío de correos se realiza con Nodemailer [11] sobre el servidor SMTP de Gmail, autenticado con una *contraseña de aplicación* de Google [13] que se guarda solo en el archivo de configuración del servidor (`.env`), fuera del repositorio. Los datos que llegan a la API se validan con **Zod** [12], una biblioteca que describe la forma esperada de cada petición y produce mensajes de error claros, en español, cuando algo no cumple.

III. Desarrollo Práctico

A. Arquitectura del proyecto

El repositorio se divide en dos aplicaciones independientes que se ejecutan juntas con un único comando (`npm run dev`):

```
ml_creaciones/
├── frontend/          React + TypeScript
│   └── src/
│       ├── api/       llamadas a la API
│       ├── components/ catálogo, pedido,
│       │              panel, auth, admin
│       ├── context/   sesión y tienda
│       ├── hooks/     useAuth, useCarga…
│       ├── pages/     una por ruta
│       └── styles/    colores de la marca
├── backend/           Node + Express
│   ├── src/
│   │   ├── auth/      registro, login,
│   │   │              códigos, sesiones
│   │   ├── pedidos/   pedidos, avisos
│   │   ├── productos/ catálogo, imágenes
│   │   ├── db.ts      tablas SQLite
│   │   └── mail.ts    plantillas de correo
│   └── data/          BD y fotos (no se
│                      sube a git)
└── docs/
```

El frontend suma 46 archivos (≈ 2 900 líneas) y el backend 21 archivos (≈ 1 650 líneas). La API expone 24 rutas agrupadas en cuatro *routers*: autenticación, productos, pedidos y presupuestos.

B. Modelo de datos

La base de datos se crea automáticamente al iniciar el servidor. La Tabla I resume sus tablas.

TABLA I. TABLAS DE LA BASE DE DATOS

Tabla	Contenido
`usuarios`	Nombre, correo único, *hash* de la contraseña, rol (admin/cliente), si está verificado y versión de sesión.
`codigos_verificacion`	Códigos para confirmar la cuenta (hasheados, con vencimiento e intentos).
`codigos_recuperacion`	Códigos para restablecer la contraseña.
`productos`	Nombre, descripción, rango de precios, stock, foto y si está visible.
`pedidos`	Cliente, producto, monto, entrega, pago, fecha de entrega y estado.
`presupuestos`	Cotizaciones pedidas por clientes o cargadas por el admin.

Los pedidos guardan una copia del nombre del producto, para que el historial no cambie si luego se edita el catálogo. Los cambios de estructura sobre tablas existentes se aplican con pequeñas migraciones que verifican primero si cada columna ya existe.

C. Módulos y roles

El sistema distingue tres tipos de acceso, protegidos tanto en el frontend como en el servidor (Tabla II).

TABLA II. PANTALLAS POR ROL

Ruta	Rol	Función
`/`	Todos	Catálogo con stock en vivo; consulta por WhatsApp.
`/registro`, `/verificar`	Visitante	Crear cuenta y confirmarla con el código.
`/login`, `/recuperar`	Visitante	Ingresar o restablecer la contraseña.
`/mis-pedidos`	Cliente	Seguir sus pedidos y presupuestos.
`/admin`	Admin	Agenda de pedidos y presupuestos.
`/admin/productos`	Admin	Crear y editar productos, stock y fotos.

En el frontend, el componente `RutaProtegida` redirige al login a quien no tiene sesión y al catálogo a quien no tiene el rol requerido. Esa protección es solo de comodidad: la seguridad real está en el servidor, donde los *middlewares* `requireAuth` y `requireRol` verifican la cookie y el rol en cada petición.

D. Registro y verificación por correo

El alta de una cuenta sigue el flujo habitual de los servicios web:

1) El visitante ingresa nombre, Gmail, contraseña (mínimo 8 caracteres) y su confirmación.
2) El servidor crea el usuario sin verificar y genera un código aleatorio de 6 dígitos con `crypto.randomInt`.

3) El código se guarda **hasheado** (HMAC-SHA256) y se envía por correo con una plantilla con los colores de la marca.
4) El usuario lo ingresa en seis casillas (admite pegarlo de una vez). Si es correcto, la cuenta queda verificada, se inicia la sesión y se muestra el saludo de bienvenida.

Cada código vence a los 15 minutos, admite 5 intentos y solo puede reenviarse cada 60 segundos. La comparación se hace en tiempo constante (`timingSafeEqual`) para no filtrar información por el tiempo de respuesta.

E. Recuperación de contraseña

La opción "¿Olvidaste tu contraseña?" reutiliza el mismo gestor de códigos. Para no revelar qué correos tienen cuenta, el servidor responde siempre lo mismo, exista o no el usuario. Al cambiar la contraseña se incrementa la columna `sesion_version`; como cada JWT lleva la versión con la que fue emitido, todas las sesiones abiertas en otros dispositivos dejan de ser válidas. Además se envía un correo avisando del cambio.

F. Pedidos y control de stock

Un cliente con sesión elige un producto del catálogo, indica el monto dentro del rango permitido, la fecha de entrega, el método de entrega (retiro o delivery) y el de pago (efectivo o transferencia). El servidor vuelve a validar todo y registra el pedido dentro de una **transacción**, descontando el stock solo si todavía hay unidades:

```
db.exec('BEGIN IMMEDIATE')
UPDATE productos SET stock = stock - 1
  WHERE id = ? AND stock > 0
-- si no cambió ninguna fila: "sin stock"
INSERT INTO pedidos (...) VALUES (...)
db.exec('COMMIT')
```

Así, aunque dos clientes pidan la última unidad al mismo tiempo, solo uno lo consigue. Cada pedido recorre los estados **pendiente → en preparación → listo → entregado**; también puede **cancelarse**, lo que devuelve la unidad al stock. El cliente puede cancelar solo mientras su pedido está pendiente; el administrador, en cualquier etapa previa a la entrega. En *Mis pedidos* el cliente ve una línea de progreso con la etapa actual.

G. Avisos por correo

Los cambios relevantes generan correos automáticos (Tabla III).

TABLA III. CORREOS AUTOMÁTICOS

Evento	Destinatario
Pedido recibido (con resumen)	Cliente
En preparación / listo / entregado / cancelado	Cliente
Nuevo pedido web	Tienda
Cliente canceló su pedido	Tienda
Pedido de presupuesto	Tienda
Código de verificación o recuperación	Usuario
Contraseña cambiada	Usuario

Los correos se envían en segundo plano: si Gmail falla, el pedido se guarda igual y el error queda registrado. En los avisos a la tienda, el campo *Responder a* apunta al cliente, para contestarle directamente. Para desarrollo existe el modo `MAIL_MODO=consola`, que muestra los correos en la terminal sin enviarlos.

H. Panel de administración

El panel tiene dos pestañas. En **Pedidos**, la agenda se filtra por estado (activos, entregados, cancelados), ordena los activos por fecha de entrega y destaca los atrasados o con entrega hoy o mañana; desde allí el administrador avanza cada pedido y carga pedidos de clientes que compran en el local. En **Productos** puede crear y editar productos, ajustar el stock con botones +/−, ocultarlos del catálogo sin borrarlos y subir una foto con vista previa.

Las imágenes se reciben con Multer, con un máximo de 5 MB. El formato se determina leyendo los primeros bytes del archivo (*magic numbers*) y no por su nombre, de modo que un archivo de texto renombrado como `.jpg` es rechazado. Cada foto se guarda con un nombre aleatorio y la anterior se elimina.

I. Medidas de seguridad

- Contraseñas con bcrypt; códigos de un solo uso guardados como HMAC.
- Sesión en cookie `HttpOnly` y `SameSite=Lax`; `Secure` en producción.
- Límite de 20 intentos cada 15 minutos por dispositivo en las rutas de autenticación.
- Mensaje genérico ante credenciales incorrectas y tiempo de respuesta similar exista o no el correo.
- Validación de todas las entradas con Zod; consultas SQL parametrizadas.
- React escapa el contenido al mostrarlo, lo que evita la inyección de HTML (XSS) que permitía el uso de `innerHTML` en la versión anterior.
- Credenciales y base de datos excluidas del repositorio público mediante `.gitignore`.

IV. RESULTADOS

El sistema cumple los requerimientos planteados. La Tabla IV compara ambas versiones.

TABLA IV. PRIMERA VERSIÓN FRENTE A LA ACTUAL

Aspecto	Versión 1	Versión 2
Tecnologías	HTML, Bootstrap, JS	React, TS, Node, SQLite
Datos	`localStorage` (un navegador)	Servidor (compartidos)
Usuarios	No	Registro, login, roles
Stock	Por navegador	Único y transaccional
Panel	Visible para todos	Solo administrador
Estado del pedido	No	5 estados + progreso
Correos	No	Automáticos
Productos	Fijos en el HTML	Editables con foto

Para verificar el funcionamiento se aplicaron tres niveles de control. Primero, el compilador de TypeScript en modo estricto y el linter `oxlint` sin errores ni advertencias. Segundo, pruebas de la API con `curl` sobre una instancia aislada del servidor —con su propia base de datos y el modo de correo en consola—, cubriendo casos válidos y de error: contraseñas que no coinciden, códigos incorrectos o

vencidos, falta de stock, montos fuera de rango, accesos sin permiso, archivos falsos o demasiado grandes. Tercero, pruebas manuales en el navegador de los flujos completos, incluyendo la recepción real de los correos en Gmail.

V. Conclusiones

Incorporar un backend resolvió el problema central de la primera versión: los pedidos, el stock y los usuarios pasaron a vivir en un único lugar, y cada persona ve solo lo que le corresponde según su rol. La separación en frontend y backend, cada uno organizado por módulos, permitió agregar funciones de forma incremental —login, pedidos, avisos, recuperación de contraseña, gestión de productos— sin reescribir lo anterior.

TypeScript y la validación con Zod en ambos extremos redujeron los errores por datos inesperados, y las transacciones de SQLite garantizaron la consistencia del stock. La verificación por correo y los avisos automáticos acercan el sistema al funcionamiento de un servicio web real, manteniendo la identidad visual de la marca.

Como trabajo futuro se propone: publicar la aplicación en un servidor con dominio propio (actualmente funciona en entorno local), integrar una pasarela de pago para tarjetas, incorporar pruebas automatizadas, permitir editar pedidos existentes, agregar un resumen de ventas para la administración y migrar a PostgreSQL si el volumen de datos lo requiere.

VI. Uso de Herramientas de IA

En el desarrollo de este sistema se utilizó el asistente de programación Claude (Anthropic), a través de Claude Code, para [describir: por ejemplo, generar y revisar código, proponer la arquitectura y redactar documentación]. El equipo [describir: definió los requerimientos, probó cada funcionalidad, revisó el código y tomó las decisiones de diseño].

VII. Referencias

[1] Meta Open Source, "React — Documentación," 2025. [En línea]. Disponible en: https://react.dev. [Accedido: oct. 2026].

[2] Microsoft, "The TypeScript Handbook," 2025. [En línea]. Disponible en: https://www.typescriptlang.org/docs/. [Accedido: oct. 2026].

[3] Vite Team, "Vite — Guía," 2025. [En línea]. Disponible en: https://vite.dev/guide/. [Accedido: oct. 2026].

[4] Bootstrap Team, "Bootstrap 5.3 — Introduction," 2024. [En línea]. Disponible en: https://getbootstrap.com/docs/5.3/. [Accedido: oct. 2026].

[5] OpenJS Foundation, "Node.js — SQLite (node:sqlite)," 2025. [En línea]. Disponible en: https://nodejs.org/api/sqlite.html. [Accedido: oct. 2026].

[6] OpenJS Foundation, "Express 5 — Guía," 2025. [En línea]. Disponible en: https://expressjs.com/. [Accedido: oct. 2026].

[7] SQLite Consortium, "SQLite Documentation," 2025. [En línea]. Disponible en: https://sqlite.org/docs.html. [Accedido: oct. 2026].

[8] M. Jones, J. Bradley y N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, may. 2015.

[9] OWASP Foundation, "Password Storage Cheat Sheet," 2025. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accedido: oct. 2026].

[10] Mozilla Developer Network (MDN), "Cookies HTTP," 2025. [En línea]. Disponible en: https://developer.mozilla.org/es/docs/Web/HTTP/Cookies. [Accedido: oct. 2026].

[11] Nodemailer, "Nodemailer — Documentation," 2025. [En línea]. Disponible en: https://nodemailer.com/. [Accedido: oct. 2026].

[12] C. McDonnell, "Zod — Documentation," 2025. [En línea]. Disponible en: https://zod.dev/. [Accedido: oct. 2026].

[13] Google, "Cómo acceder con contraseñas de aplicación," 2025. [En línea]. Disponible en: https://support.google.com/accounts/answer/185833. [Accedido: oct. 2026].

Profesor: Carlos A. Benítez B. | *Quinto Semestre* | *Caacupé – Paraguay, 2026*